SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME:Artificial Intelligence **COURSE CODE: CSA1701**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I ___**shaikThouhid(192421280)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

thouhid

Question 1: Hospital Shift Scheduling via Backtracking Search

Problem Formulation

- **Variables:** Doctors D1, D2, D3
- **Domains:** Shifts {Morning (M), Afternoon (A), Night (N)} mapped chronologically as M < A < N
- **Constraints:**
  1. D1 ≠ Night
  2. Shift(D2) < Shift(D3)
  3. All shifts distinct (One doctor per shift, bijection between {D1, D2, D3} and {M, A, N})
  4. D3 ≠ Morning
  5. Every doctor assigned exactly one shift

Extended Explanation: Why this is a Constraint Satisfaction Problem (CSP)

A CSP is defined by three components: a set of variables X, a set of domains D (one per variable), and a set of constraints C that restrict which combinations of values are allowed. Here X = {D1, D2, D3}, every domain is {M, A, N}, and C contains the five rules above. A solution is a *complete* assignment (every variable has a value) that is also *consistent* (no constraint is violated).

The constraints are of two kinds:

- **Unary constraints** involve one variable: D1 ≠ N and D3 ≠ M. These can be applied before the search starts by shrinking domains: Dom(D1) = {M, A}, Dom(D3) = {A, N}, Dom(D2) = {M, A, N}.
- **Binary constraints** involve two variables: Shift(D2) < Shift(D3), and the all-different constraint (D1 ≠ D2, D1 ≠ D3, D2 ≠ D3).

Constraint graph: D1, D2 and D3 are all connected to each other (because of the alldifferent rule), and D2 – D3 additionally carries the ordering constraint. Because the graph is a complete triangle, no variable can be solved independently of the others.

Extended Explanation: The Backtracking Algorithm

Backtracking search is depth-first search over partial assignments. At each level it picks one unassigned variable, tries every value in its domain, checks consistency with the assignments made so far, and recurses. If no value works, it returns failure and the previous variable tries its next value. This early rejection (pruning) is what makes backtracking far better than blind generate-and-test, which would build all $3^3 = 27$ assignments before checking anything.

```
function BACKTRACK(assignment, csp):     if
assignment is complete: return assignment
var ← SELECT-UNASSIGNED-VARIABLE(csp)
for each value in ORDER-DOMAIN-VALUES(var):
if value is consistent with assignment:
add {var = value} to assignment
result ← BACKTRACK(assignment, csp)
if result ≠ failure: return result
```

```
remove {var = value} from assignment
return failure
```

Step-by-Step Backtracking Search Execution

1. **Step 1 — Assign Shift to D1:**
   - Try D1 = Morning (M):
     - Check Constraints: D1 ≠ Night (Satisfied).
     - Remaining available shifts for D2, D3: {Afternoon (A), Night (N)}.
     - 2. **Step 2 — Assign Shift to D2 (given D1 = M):**
   - Try D2 = Afternoon (A):
     - Check Constraints: Distinct shift from D1 (Satisfied). Remaining available shift for D3: {Night (N)}.
     - 3. **Step 3 — Assign Shift to D3 (given D1 = M, D2 = A):**
   - Try D3 = Night (N):
     - Check Constraint iv (D3 ≠ M): Satisfied (N ≠ M).
     - Check Constraint ii (D2 < D3): A < N is TRUE (Afternoon precedes Night).
     - Check Constraint iii (All distinct): {M, A, N} are all distinct.
     - All constraints satisfied! Complete valid assignment found.
     -

Extended Explanation: Full Search Tree (all branches)

To show the search is exhaustive, here is every branch the algorithm would explore if it continued after finding the first solution (values tried in the order M, A, N):

Depth 1 (D1)	Depth 2 (D2)	Depth 3 (D3)	Outcome
D1 = M	D2 = M	–	Rejected: D2 = D1 (not distinct)
D1 = M	D2 = A	D3 = M	Rejected: D3 = Morning violates constraint 4 (also not distinct)
D1 = M	D2 = A	D3 = A	Rejected: not distinct
D1 = M	D2 = A	D3 = N	**SOLUTION 1**
D1 = M	D2 = N	D3 = A	Rejected: N < A is false
D1 = A	D2 = M	D3 = N	**SOLUTION 2**

D1 = A	D2 = N	D3 = M	Rejected: D3 = Morning and N < M is false
D1 = N	–	–	Pruned immediately: D1 ≠ Night violated

Only **2 of the 27** possible assignments are valid, and backtracking visits only a small fraction of the full space.

Backtracking Tree Evaluation & Verification Table

Branch / Path	Assignments	Constraint Checks	Status
Path 1 (Target)	D1=M, D2=A, D3=N	D1≠N (OK), D3≠M (OK), D2 < D3 (A < N: OK), All Distinct	**VALID (Solution)**
Path 2 (Alt Branch)	D1=M, D2=N, D3=A	D2 < D3 fails (Night does not precede Afternoon)	BACKTRACK
Branch / Path	Assignments	Constraint Checks	Status
Path 3 (Alt Branch)	D1=A, D2=M, D3=N	D1≠N (OK), D3≠M (OK), D2 < D3 (M < N: OK), All Distinct	**VALID (Solution)**
Path 4 (Pruned)	D1=N, ...	D1 ≠ Night violated immediately at depth 1	PRUNED

Final Valid Hospital Shift Schedule

```
==================================================
FINAL SHIFT ASSIGNMENT:
--------------------------------------------------
•   Morning Shift   (M) : Doctor D1• Afternoon
Shift (A) : Doctor D2
•   Night Shift     (N) : Doctor
D3==================================================
===
(Note: Alternative valid assignment: D1 = Afternoon, D2 = Morning, D3 =
Night)
```

Extended Explanation: Improvements and Python Implementation

- **Forward checking:** after assigning D1 = M, remove M from the domains of D2 and D3 immediately. If any domain becomes empty, backtrack at once.

- **MRV (Minimum Remaining Values):** choose the variable with the smallest legal domain first. After the unary constraints, D1 and D3 have two values each while D2 has three, so D1 or D3 would be chosen before D2.
- **Constraint propagation (arc consistency):** for the constraint D2 < D3, D2 can never be Night and D3 can never be Morning, so Dom(D2) shrinks to {M, A} before the search begins.
- **Complexity:** worst case $O(d^n) = 3^3 = 27$ assignments; in practice far fewer are generated because of pruning.

```
from itertools import permutations

ORDER = {"M": 0, "A": 1, "N": 2}
SHIFTS = ["M", "A", "N"]

def consistent(assign):
    d1, d2, d3 = assign.get("D1"), assign.get("D2"), assign.get("D3")
    if d1 == "N": return False                                    #
constraint
1
    if d3 == "M": return False                                    #
constraint
4
    if d2 and d3 and not ORDER[d2] < ORDER[d3]: return False      #
constraint
2
    vals = [v for v in (d1, d2, d3) if v]
    return len(vals) == len(set(vals))                            #
constraint
3

def backtrack(assign, vars_, solutions):
if len(assign) == len(vars_):
solutions.append(dict(assign)); return
var = vars_[len(assign)]     for val in
SHIFTS:          assign[var] = val
if consistent(assign):
backtrack(assign, vars_, solutions)
del assign[var]

solutions = []
backtrack({}, ["D1", "D2", "D3"], solutions)
print(solutions) # [{'D1': 'M', 'D2': 'A', 'D3': 'N'}, {'D1': 'A',
'D2': 'M', 'D3': 'N'}]
```

Conclusion for Question 1: The problem has exactly two valid schedules. Both give the Night shift to D3, which is forced because D3 cannot work Morning and must work later than D2.

Question 2: 5×5 Grid Navigation via Best-First / A* Search

Environment Setup & Specifications

- Grid Size: 5 × 5 coordinates (0..4, 0..4)
- Start (S): (0, 0) | Goal (G): (4, 4)
- Obstacles (X): (1, 1), (1, 2), (2, 3), (3, 1)
- Heuristic Function: Manhattan Distance h(x, y) = |x - 4| + |y - 4|
- Step Cost: g(node) = 1 per orthogonal step
- Algorithm Evaluation Function: f(n) = g(n) + h(n) [A* Search / Informed Best-First]
- **Extended Explanation: How A* Works**
- A* keeps two lists. The **Open list** (a priority queue) holds nodes that have been generated but not yet expanded, ordered by the lowest f(n). The **Closed list** holds nodes already expanded. At every step A* removes the node with the smallest f from Open, tests whether it is the goal, and otherwise generates its legal neighbours (up, down, left, right, inside the grid and not an obstacle). Each neighbour gets g = parent's g + 1 and f = g + h. If a neighbour is reached again by a cheaper route, its g and parent are updated.

- g(n) is the exact cost already paid from the start to n. h(n) is the estimated cost still to
- pay from n to the goal. f(n) is the estimated cost of the cheapest solution that passes
- through n.

- **Why Manhattan distance is a good heuristic here:** the robot moves only in four directions with unit cost, so the true remaining cost can never be less than the horizontal plus vertical distance. This makes h **admissible** (it never overestimates). It is also **consistent**, because moving one step changes h by exactly 1 and the step cost is 1, so $h(n) \leq 1 + h(n')$. With an admissible and consistent heuristic, A* is complete and returns an optimal path, and a node never needs to be re-expanded.

Visual Representation of 5×5 Grid

	Col 0	Col 1	Col 2	Col 3	Col 4
0	[S]	[ ]	[ ]	[ ]	[ ]
1	[ ]	[X]	[X]	[ ]	[ ]
2	[ ]	[ ]	[ ]	[X]	[ ]
3	[ ]	[X]	[ ]	[ ]	[ ]
4	[ ]	[ ]	[ ]	[ ]	[G]

Step-by-Step Priority Queue Trace

Step	Expanded Node	Successors Generated (x, y)	f(n) = g(n) + h(n)	Priority Queue / Open List (Sorted by f)
0	Init (0,0)	–	g=0, h=8, f=8	[(0,0): f=8]
Step	Expanded Node	Successors Generated (x, y)	f(n) = g(n) + h(n)	Priority Queue / Open List (Sorted by f)
1	(0,0)	(1,0), (0,1)	(1,0): g=1, h=7, f=8; (0,1): g=1, h=7, f=8	[(1,0): f=8, (0,1): f=8]
2	(1,0)	(2,0) [Note: (1,1) is Obstacle]	(2,0): g=2, h=6, f=8	[(0,1): f=8, (2,0): f=8]
3	(2,0)	(3,0), (2,1)	(3,0): g=3, h=5, f=8; (2,1): g=3, h=5, f=8	[(0,1): f=8, (3,0): f=8, (2,1): f=8]
4	(2,1)	(2,2) [Note: (3,1),(1,1) Obstacles]	(2,2): g=4, h=4, f=8	[(2,2): f=8, (0,1): f=8, (3,0): f=8]
5	... (2,2) -> (3,2) -> (4,2) -> (4,3) -> (4,4)	Progresses directly toward G with minimal f(n)	–	[(4,4): f=8]

Extended Explanation: Detailed Expansion of Step 5 Onward

Step 5 in the table summarises the last part of the search. Expanding it one node at a time (ties on f are broken in favour of the deeper node, i.e. larger g, which is a common A* tiebreak):

Step	Expanded Node	Successors Generated	f values	Notes
5	(2,2)	(3,2)	(3,2): g=5, h=3, f=8	(1,2) and (2,3) are obstacles; (2,1) is already closed

6	(3,2)	(4,2), (3,3)	(4,2): g=6, h=2, f=8; (3,3): g=6, h=2, f=8	(3,1) is an obstacle
7	(4,2)	(4,3), (4,1)	(4,3): g=7, h=1, f=8; (4,1): g=7, h=5, f=12	(4,1) has a high f so it stays low in the queue
8	(4,3)	(4,4)	(4,4): g=8, h=0, f=8	Goal generated
Step	Expanded Node	Successors Generated	f values	Notes
9	(4,4)	–	–	Goal removed from Open and tested: **search stops**

Notice that f stays equal to 8 along the entire optimal route. This is exactly what we expect when h is a perfect lower bound: the heuristic never overestimated, so the optimal cost (8) equals the initial h-value of the start node (8). Nodes such as (0,1) and (3,0) were generated with f = 8 but never needed to be expanded, because the tie-break pushed the search along the deeper branch toward the goal.

Optimal Path & Total Movement Cost

Optimal Sequence of Grid Coordinates:

(0, 0) → (1, 0) → (2, 0) → (2, 1) → (2, 2) → (3, 2) → (4, 2) → (4, 3) → (4, 4) **Cost**

Calculation:

- Total Moves = 8 steps
- Cost per Move = 1
- Total Optimal Path Cost = 8 units
-

Extended Explanation: Why 8 is Optimal and Other Equal-Cost Paths

- The Manhattan distance from (0,0) to (4,4) is 8, so no path can be shorter than 8 moves. Since a path of exactly 8 moves exists, it is optimal.
- Other optimal paths also exist, for example (0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4), which also costs 8. A* returns whichever the tie-breaking rule reaches first.
- Any path that moves left or up (away from the goal) would cost at least 10, since every backward step must be undone by one extra forward step.

Extended Explanation: Python Implementation

```
import heapq

START, GOAL = (0, 0), (4, 4)
OBSTACLES = {(1, 1), (1, 2), (2, 3), (3, 1)}


def h(n):
    return abs(n[0] - GOAL[0]) + abs(n[1] - GOAL[1])

def astar():     open_list = [(h(START), 0, START, None)]   #
(f, g, node, parent)
    best_g, parent, closed = {START: 0}, {},
set()     while open_list:         f, g,
node, par = heapq.heappop(open_list)
if node in closed:
            continue
        closed.add(node); parent[node]
= par         if node == GOAL:
path = []
            while node: path.append(node); node = parent[node]
return path[::-1], g         x, y = node         for nx, ny in
((x+1, y), (x-1, y), (x, y+1), (x, y-1)):             nb = (nx,
ny)             if 0 <= nx <= 4 and 0 <= ny <= 4 and nb not in
OBSTACLES:                 ng = g + 1                 if ng <
best_g.get(nb, float("inf")):                     best_g[nb] =
ng
                    heapq.heappush(open_list, (ng + h(nb), ng, nb,

node))     return None, float("inf") print(astar())   # an 8-move

path, cost 8
```

Extended Explanation: A* Compared with Other Strategies

Strategy	Uses	Optimal?	Behaviour on this grid
BFS (uninformed)	g only (depth)	Yes (unit costs)	Expands nearly every reachable cell before reaching the goal
Greedy Best-First	h only	No	Fast, but can follow a misleading heuristic into a dead end

A*	g + h	**Yes** (admissible h)	Expands only nodes with f = 8 along and near the optimal route

---

Question 3: Autonomous Disaster Rescue Robot Agent

Part a) Analysis of Constraints on Decision-Making

1. **4-Directional Movement:** Restricts action space to {Up, Down, Left, Right}, dictating grid-based successor generation.
2. **Obstacle Blocks:** Introduces hard environmental constraints; blocked cells cannot be expanded, requiring dynamic path re-routing.
3. **Partial Observability (Adjacent Sensing Only):** The agent operates with an incomplete map. Unvisited cells are unknown, preventing full offline path computation upfront and necessitating local state discovery.
4. **Variable Path Costs (Regular = 1, Risky = 3):** Differentiates safe traverse (cost = 1) from high-risk structural zones (cost = 1 + 2 = 3). Forces the agent to balance path distance against traversal risk.
5. **Dynamically Updated Knowledge (Online Search):** The agent must interleave planning and execution, updating its internal map representation upon each perception step.

Extended Explanation of Part a)

- **Effect on the action space:** with only four moves, the branching factor is at most 4 (and usually 3 after excluding the cell just visited). The agent cannot cut diagonally, so the shortest route is measured in Manhattan-style steps.
- **Effect of obstacles:** a blocked cell is treated as having infinite cost. When a new obstacle is sensed on the current route, the old plan becomes invalid and the agent must re-plan from where it stands, so the agent cannot commit to a long plan in advance.
- **Effect of partial observability:** the agent knows only the cells it has sensed. It must keep an internal model (a map memory) of everything seen so far and treat unseen cells as unknown, which creates an *exploration versus exploitation* trade-off: explore to learn the map, or head straight for the goal.
- **Effect of variable costs:** a risky cell costs 3, so a detour of up to 2 extra regular steps is cheaper than crossing one risky cell. For example, a straight route of 4 steps containing one risky cell costs 3 + 3 = 6, while a safe detour of 6 regular steps costs 6; at 7 detour steps the risky route becomes better. The agent must compare total cost, not just distance.
- **Effect of dynamic knowledge:** the map the agent holds is only a belief that improves with every percept, so the decision at each step uses the newest information.

Part b) Proposed Solution Approach: LRTA* / Online Uniform Cost Search

Selected Search Framework: Learning Real-Time A* (LRTA*) with Dynamic Cost Mapping (Online UCS).

Algorithmic Execution Steps:

1. **Perception Phase:** Sense adjacent cells (Up, Down, Left, Right) from current position s. Identify walls, open paths, and risky zones.
2. **Knowledge Base Update:** Update local grid map with observed cell traversal costs: regular = 1, risky = 3, obstacle = ∞.
3. **Heuristic / Cost Evaluation:** For each adjacent non-blocked neighbour s', calculate estimated total cost f(s') = g(s') + h(s').
4. **Action Selection:** Move to the neighbouring cell s' that minimizes the estimated remaining cost.
5. **Value Learning & Loop:** Update the heuristic cost estimate of current state s based on neighbour values: H(s) ← max(H(s), min_s' [c(s, s') + H(s')]). Repeat until Goal G is reached.

Extended Explanation of Part b) **Pseudocode**

of the agent loop:

```
H ← table of heuristic estimates (initially Manhattan distance to G)
map ← empty knowledge base
s ← start cell while s ≠ G:     percept ← SENSE(s)
# up, down, left, right cells
    UPDATE(map, percept)                    # cost 1, cost 3, or ∞
(obstacle)     for each non-blocked neighbour s' of
s:         score(s') ← c(s, s') + H(s')         # c
= 1 or 3     H(s) ← max(H(s), min over s' of
score(s'))     a ← action leading to the s' with the
lowest score     s ← MOVE(a)
```

Small worked example of one decision. Suppose the robot stands at cell s with H(s) = 5. It senses two usable neighbours: Up is a regular cell (c = 1) with H = 4, and Right is a risky cell (c = 3) with H = 1.

Neighbour	Cell type	Step cost c	H(s')	c + H(s')
Up	Regular	1	4	5
Right	Risky	3	1	4

Right has the lowest estimate (4), so the robot moves Right even though the cell is risky, because the overall remaining cost is lower. Then H(s) ← max(5, 4) = 5. The robot keeps H(s) from dropping, which is valid when the starting heuristic is admissible. If the neighbour of Up later turns out to lead into a dead end, its H value is raised, which stops the robot from looping back.

Why "Online UCS" is mentioned alongside LRTA*: Uniform Cost Search orders nodes only by the path cost g, which uses the true traversal costs (1 or 3) as they are sensed. LRTA* adds a learned heuristic H on top, so the robot is guided toward the goal rather than spreading out evenly in all directions.

Part c) Demonstration of Core AI Concepts

AI Concept	Theoretical Framework	Application in Rescue Robot Scenario
Intelligent Agent	PEAS Paradigm (Performance, Environment, Actuators, Sensors)	**P:** Maximize survivor safety, minimize cost. **E:** Grid building with collapse zones. **A:** Wheeled/tracked motors (4-way). **S:** Adjacent distance/risk sensors.
Rationality	Expected Utility Maximization	The robot makes optimal local choices based on current percept history, choosing low-risk paths over shorter high-risk paths when total expected cost is lower.
Environment Type	Task Environment Classification	Partially Observable (senses 1-hop); Dynamic / Stochastic (structural risks); Sequential (actions affect future states); Discrete (grid rooms)

Extended Explanation of Part c)

PEAS in more detail

- **Performance measure:** number of survivors reached, total path cost (risk plus distance), time taken, and zero robot damage or structural triggers.
- **Environment:** a collapsed building modelled as a grid, containing walls and debris (obstacles), unstable floors (risky cells), and one or more survivors (goals). •**Actuators:** drive motors for the four movement directions, and possibly a signalling device to report the survivor's location.
- **Sensors:** proximity and risk sensors for the four adjacent cells, a position sensor (odometry or localisation), and optionally a camera or thermal sensor to detect survivors.

Agent type: this robot is best described as a *utility-based agent with a model*, because it keeps an internal map (model-based) and picks actions by comparing costs and risks (utility-based), rather than reacting with simple condition-action rules.

Rationality: a rational agent selects the action expected to maximise its performance measure given its percept history and built-in knowledge. Rationality is not omniscience: the robot may still step on an unseen risky cell, but its decision was rational given what it could sense.

Environment properties (what each term means here)

Property	Value	Reason
Observability	Partially observable	Only the four adjacent cells are sensed
Agents	Single agent	One robot; no competing agents
Determinism	Stochastic / dynamic	Structures may collapse and change costs while the robot moves
Episodic vs sequential	Sequential	Each move changes the position and so affects later choices
Static vs dynamic	Dynamic	The environment can change while the robot is deciding
Discrete vs continuous	Discrete	Grid cells and four fixed actions
Property	Value	Reason
Known vs unknown	Unknown (initially)	The map must be learned during exploration

Part d) Justification of the Selected Search Strategy

- *Infeasibility of Standard Offline Search (A / Dijkstra):** Standard A* requires complete global knowledge of the graph prior to execution. In a partially observable disaster environment, the robot cannot construct an offline plan because unmapped obstacles and unknown risky zones would invalidate pre-computed paths.
- *Advantage of Online Search (LRTA / Dynamic UCS):** Online search interleaves computation with real-time sensing and movement. It guarantees completeness in finite state spaces and avoids infinite loops by updating state memory (learning), ensuring safe, optimal, and adaptive navigation toward survivor G.

Extended Explanation of Part d)

Offline versus online search. An offline algorithm computes the whole solution first and then executes it without looking at the world again. An online algorithm computes only the next action, executes it, observes the result, and repeats. A disaster site is unknown in advance, so only the online approach is usable.

Feature	A* / Dijkstra (offline)	LRTA* / Online UCS
Needs full map beforehand	Yes	No

Plans before moving	Entire path	Only the next move
Reacts to newly sensed obstacles	Must restart planning	Updates its map and continues
Memory used	Open and closed lists	Map plus H table (learned values)
Risk of looping	None (with a closed list)	Avoided by raising H of revisited states
Suitable for this scenario	No	**Yes**

*Why LRTA does not loop forever.** Each time the robot leaves a cell, that cell's H value is raised to reflect what was learned about it. A cell that kept leading the robot into a dead end becomes progressively less attractive, so the robot is pushed to try other directions.
In a finite, safely explorable space the robot is guaranteed to reach the goal.

Handling risk. Because the step cost of a risky cell is 3, the cost comparison automatically makes the robot prefer a safe detour when that is cheaper. If safety must be weighted more strongly, the risky cost can simply be raised (for example from 3 to 5) without changing the algorithm.

Limitations and possible improvements. LRTA* may take a longer route on its first attempt because it learns as it moves; repeated runs on the same map improve its H values and the path converges towards the optimal one. For larger or changing maps, D* Lite is a related incremental online algorithm that repairs its plan efficiently when new obstacles are discovered.

Overall Conclusion

- **Question 1** is solved by backtracking on a CSP: unary constraints and early pruning reduce 27 possible assignments to exactly 2 valid schedules.
- **Question 2** is solved by A* with the Manhattan heuristic, which is admissible and consistent; the 8-move path found is provably optimal.
- **Question 3** requires online search, because the rescue robot is in a partially observable, dynamic environment; LRTA* with dynamic cost mapping lets it sense, learn and move safely toward the survivor.